生成式AI Final project document

第十六組

楊宇翔 **412261377**

詹恩碩 **412261559**

鍾慶玲 **412261248**

林恒箴 **412261092**

1.Background Knowledge

身為大學生，看動漫是我們許多人的日常娛樂。但在這個資訊爆炸的時代，MyAnimeList 上面登錄的作品成千上萬，要在這麼龐大的資料庫裡找到一部對味的作品其實很花時間。我們使用的資料集來自 Kaggle 的 "MyAnimeList Anime Dataset 2025"，雖然資料很豐富，但我們發現現有的分類標籤其實很籠統。

此外，近期 Llama 等開源模型表現越來越強，讓我們萌生了「能不能訓練一個 AI 專門來推薦動漫？」的想法。然而，我們在初步嘗試時發現了個大問題：AI 會亂講話，也就是幻覺，通用模型常會一本正經地推薦根本不存在的作品，而且格式都會亂跑。

這促使我們決定結合 LoRA 微調技術 與 RAG 架構，試著用比較低的運算成本4-bit 量化，訓練出一個聽話的 AI動漫 推薦員。

2.Motivation

我們做這個題目的動機，就是希望 AI 能聽得懂像是推薦一部熱血籃球番（當然我們這次是用英文的prompt）這種比較細膩的要求，而不只是丟給我一堆運動類動畫。

我們不希望 AI 變成單純的聊天機器人，而是希望它成為一個精準的檢索工具。

透過 SFT，我們強制讓AI 學會看到關鍵字就要輸出特定格式的 JSON List，而不是寫作文。

然後再去結合 RAG，也就是 AI 負責理解需求，真正的資料（如評分、年份）則回去資料庫抓。

總之呢，我們final project做這個的目的，就是想結合我們對動漫的熱愛以及 LLM 技術，做出一個真的能用、不會亂推薦假片，而且比傳統搜尋引擎更聰明的動漫推薦工具。

3. Dataset and Data Source

本專案的原始資料來自 Kaggle 開源數據集 "MyAnimeList Anime Dataset 2025"。該數據集爬取自知名動漫社群網站 MyAnimeList (MAL)，包含了大量動畫作品的詳細數據。網址是：[Kaggle - MyAnimeList 2025]

檔名：`mal_anime.csv`

原始資料大約20000筆，包含作品標題 (`title`)、類型 (`genres`)、製作公司 (`studios`)、評分 (`score`)、收藏數 (`favorites`)、簡介 (`description`) 以及發行年份 (`released_year`) 等欄位。

資料的預處理

原始數據集雖然龐大，但包含許多極其冷門或資訊不全的條目，直接使用會導致模型學習到低質量的推薦內容。因此，我們在 `1_prepare_data.py` 中實施了以下資料清洗策略：

1. 刪掉太冷門的，為了確保推薦系統的實用性與大眾接受度，我們設定了 `MIN_FAVORITES = 500` 的門檻。系統會自動過濾掉收藏數（Favorites）低於 500 的冷門作品，專注於具備一定討論度的動畫。

2. 數據格式正規化，針對數值欄位（如 Favorites），移除逗號並轉換為整數格式；同時處理年份欄位，透過正規表達式提取準確的發行年份，以便後續進行「新/舊番」的條件篩選。

3. 把缺失太多的刪掉，剔除缺失標題 (`Title`) 或缺乏關鍵描述的作品，確保每一筆訓練資料都具備完整的檢索資訊。

隱藏標籤挖掘

為了突破傳統 Genre（像是 action sports等）分類過於籠統的限制，我們實作了一套基於規則的 Text Mining 機制。透過分析 `description`（文本，我們挖掘出資料庫中未明確標示但用戶可能感興趣的隱藏標籤：

像是球類運動，能夠區分 Basketball、Soccer 、Volleyball 等具體運動項目，而非僅輸出 generic 的 "Sports" 標籤。

還有風格與題材挖掘，透過關鍵字匹配，自動標註如高階語意標籤。

如次一來程式就能自動從簡介中識別出相關作品，大幅增強了模型對特定自然語言查詢的理解能力。

SFT 訓練**data set**建構

為了讓 Llama 3.2 模型能夠進行 Instruction Tuning 並穩定輸出 JSON 格式，我們將清洗後的資料轉換為對話式數據集 `train_selector_v13_fav.jsonl`:

樣本規模 總計生成 10,000 筆對話樣本。

多樣化提示：訓練資料包含三種模式，以訓練模型應對不同類型的用戶查詢：

1. Genre Mode: 包含一般推薦、舊番推薦 (2015以前)與新作推薦 (2015後)。

2. Studio Mode: 針對特定製作公司（如 Kyoto Animation, Ghibli）的推薦請求。

3. Hidden Tag Mode: 針對上述挖掘出的特殊標籤（像是 "Recommend cyberpunk anime"）進行強化訓練。

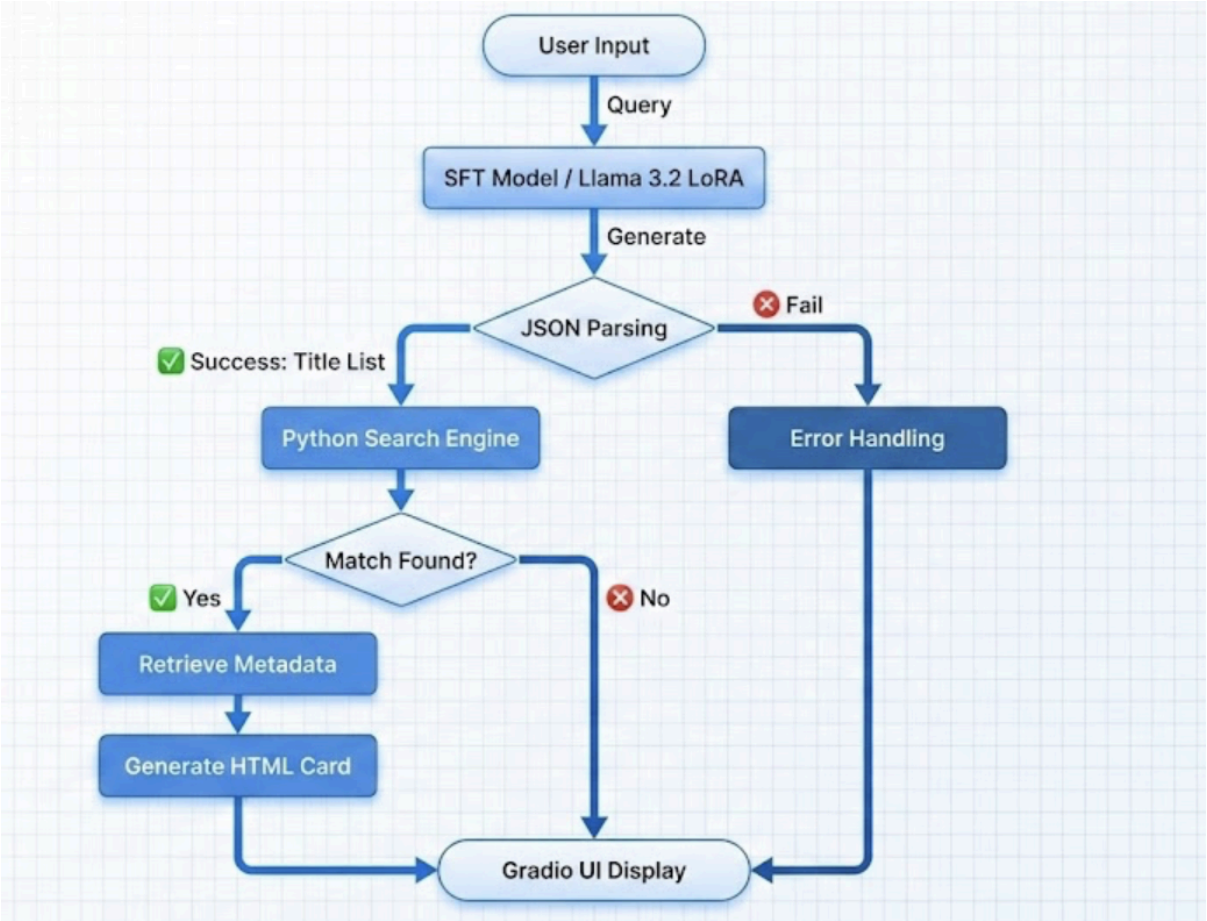
然後是輸出格式約束: 每一筆資料的 Assistant 回覆皆被嚴格格式化為純 JSON List（例如 `["Title A", "Title B", "Title C"]`），這是實現 RAG 系統穩定檢索的關鍵。

最後是資料分割: 最終數據集按 95% 訓練集 與 5% 驗證集 進行劃分，用於監控模型收斂情況。

4.Method and Flowchart

我們這組在簡報上用的是tinyllama版本，document決定改進loss曲線和模型訓練，不過我認為簡報的版本對我們這組來說也很有意義，所以簡報的版本就保留下來。我們也從原本colab改爲本地訓練，顯卡用的是rtx3080ti 12gb。

然後我們的流程圖：



架構

首先是模型推論，會先接收使用者輸入 。然後我們的SFT Model (Llama 3.2 LoRA) 負責理解語意，並嘗試生成結構化的 JSON 格式。

再來是格式驗證，系統會進入 "JSON Parsing" 判斷節點。這是本系統與 Base Model 的分水嶺：SFT 模型能成功解析並輸出 Title List (走左側路徑)；未經訓練的模型則常因不遵尋格式導致解析失敗，進入 Error Handling (走右側路徑)。

最後才是檢索與渲染：Python Search Engine 根據解析出的片名查詢資料庫。

若 "Match Found"，則提取圖片與 metadata 生成 HTML 卡片；若無資料則過濾，確保使用者不會看到錯誤資訊。

SFT 訓練部分

為了讓模型能夠穩定通過上述的 "JSON Parsing" 節點並具備動漫領域知識，我們採用了以下SFT策略：

1.模型選擇與優化

我們選的Base Model是Llama-3.2-3B-Instruct。選擇此模型是因為它在輕量級參數下具備優異的指令理解能力。

我們也用了Quantization也就是4-bit 量化: 為了在有限的顯存（我用3080ti vram僅有12gb)資源下進行訓練，我們使用 `bitsandbytes` 將模型量化為 NF4 格式，大幅降低記憶體需求同時保持推論精度。

2. LoRA

我們採用 LoRA 技術進行高效微調。

Target Modules: 針對 Transformer 架構中的 Attention Layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`) 進行權重更新。

Parameters則 設定 Rank (`r`) = 16, Alpha (`lora_alpha`) = 32。

好處是LoRA 僅需訓練極少量的參數 (約總參數量的 1% 以下)，能有效避免災難性遺忘，保留模型原本的語言能力。

3.兩大訓練目標（這邊我們想了很久）

本次 SFT 的訓練data set 設計旨在達成兩個目的：

1. 格式要對: 強制模型在回答 "Recommend..." 指令時，必須輸出純淨的 JSON List (e.g., `["Title A", "Title B"]`)，這解決了 Base Model 廢話過多的問題。

2. 隱性知識注入: 透過資料挖掘技術，我們將data set的Description中的隱藏特徵（就像是棒球元素出現於非運動分類的作品）轉化為訓練樣本，讓模型學會比關鍵字搜尋更深層的內容理解。

實作與整合

訓練完成的 Adapter 權重被整合至 Gradio 前端介面中。我們用了 Python 中介層來處理 RAG 邏輯：

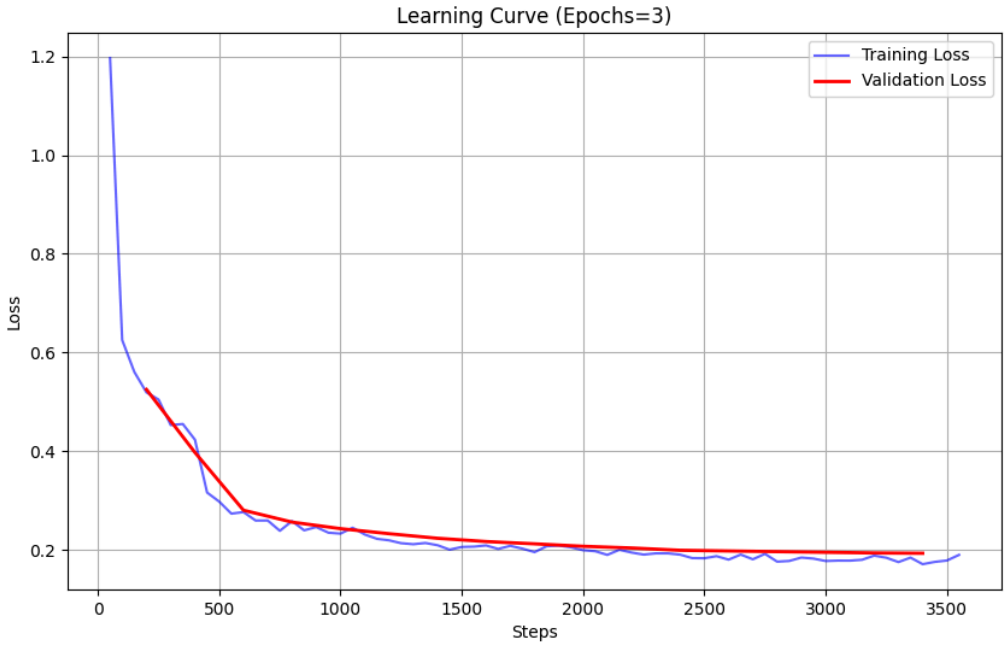
Python Search Engine: 不依賴模型的記憶來回答細節，而是將模型視為「索引器」，實際的年份、評分與圖片皆即時從 `mal_anime.csv` 讀取，確保資料的 100% 正確性。

Error Handling: 即使 SFT 模型表現穩定，系統仍設計了例外處理機制，防止任何非預期的輸出導致Crash，提升系統的Robustness。

最後以上的code皆會上傳至tronclass作業區

5. Experiments result

首先是**LOSS**曲線



可以看到前期就收斂很快，後期慢慢收斂至0.2，沒有overfitting的問題

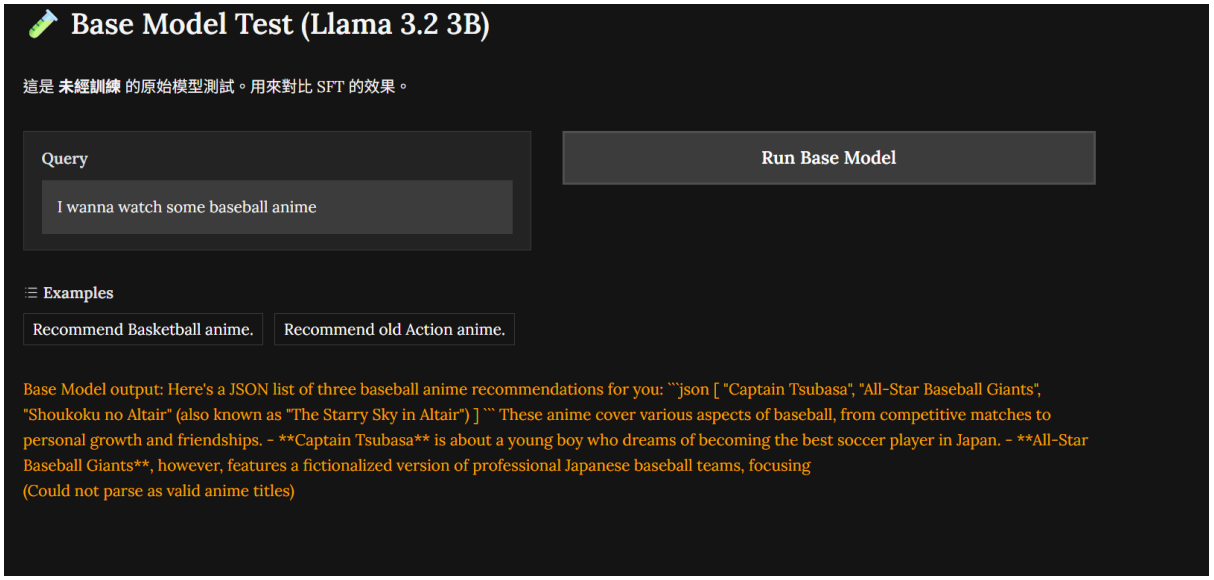
微調前後比較(為了公平對比，兩者都有用檢索)

查棒球

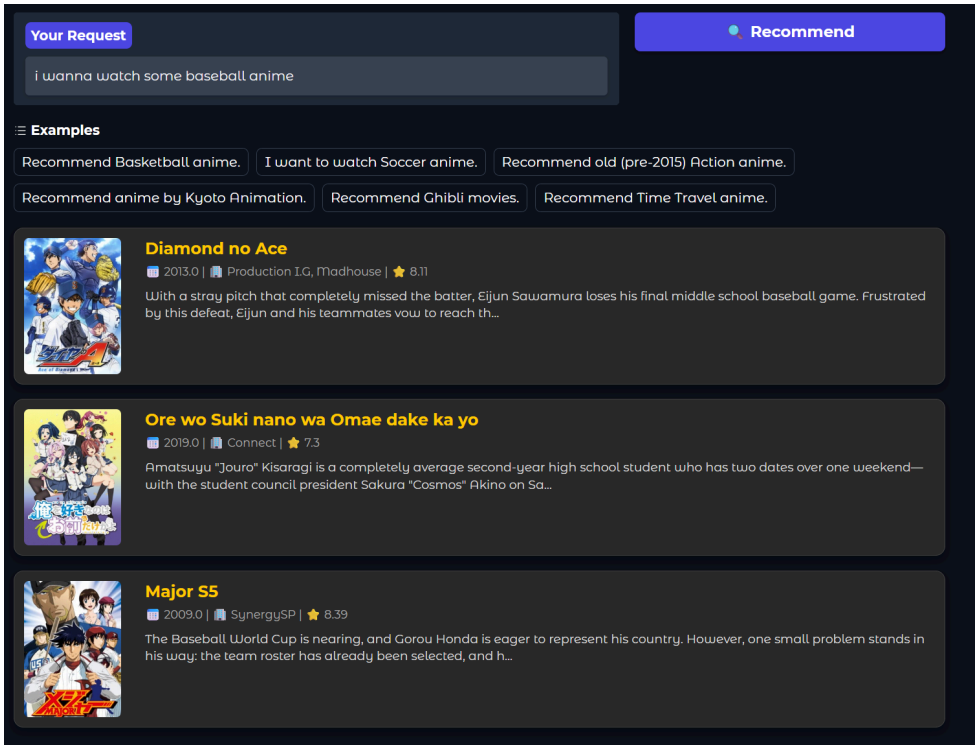
prompt:I wanna watch some baseball anime

微調前的BASE MODEL：格式錯誤跑出橘字而沒有生成**card**、答案跟棒球豪無相關：**All-Star Baseball Giants**完全在亂掰

Shoukoku那個已經在胡言亂語了

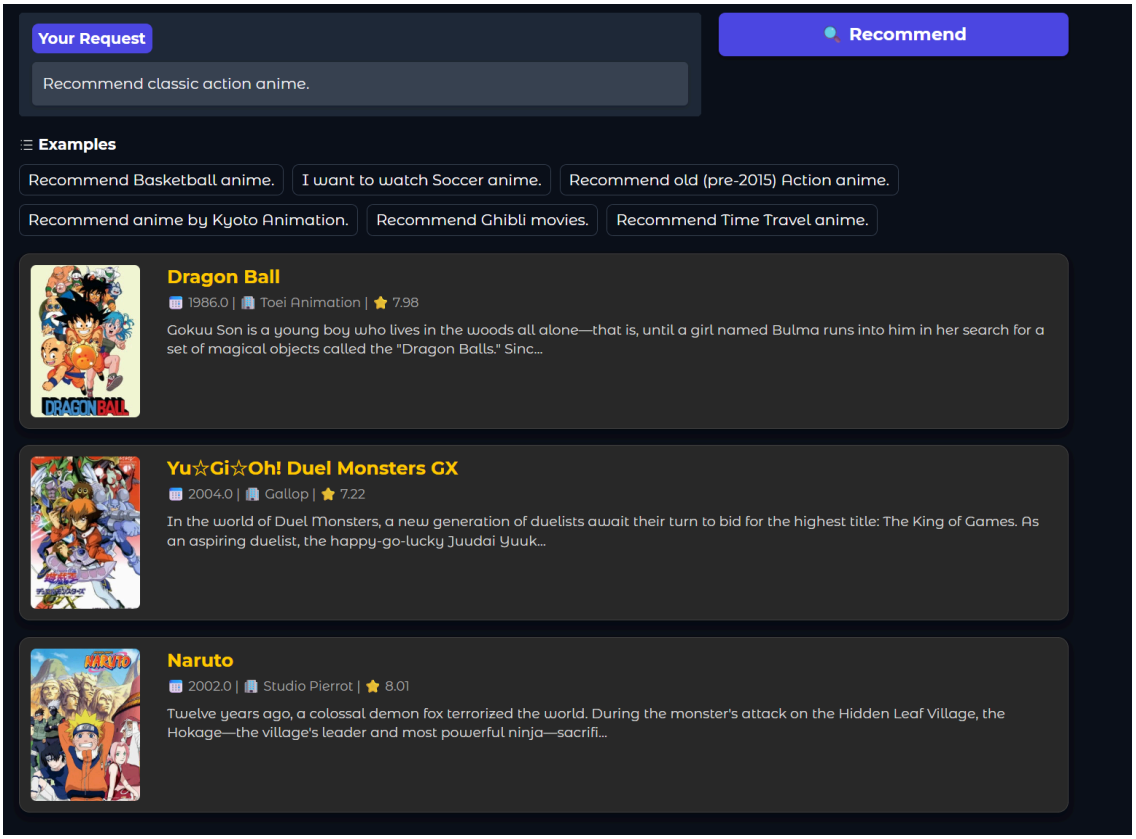


微調後的模型:格式與內容皆正確(中間那部出現的原因是裡面有棒球相關情節，但主要是校園，在SFT訓練目標有解釋隱性知識注入)

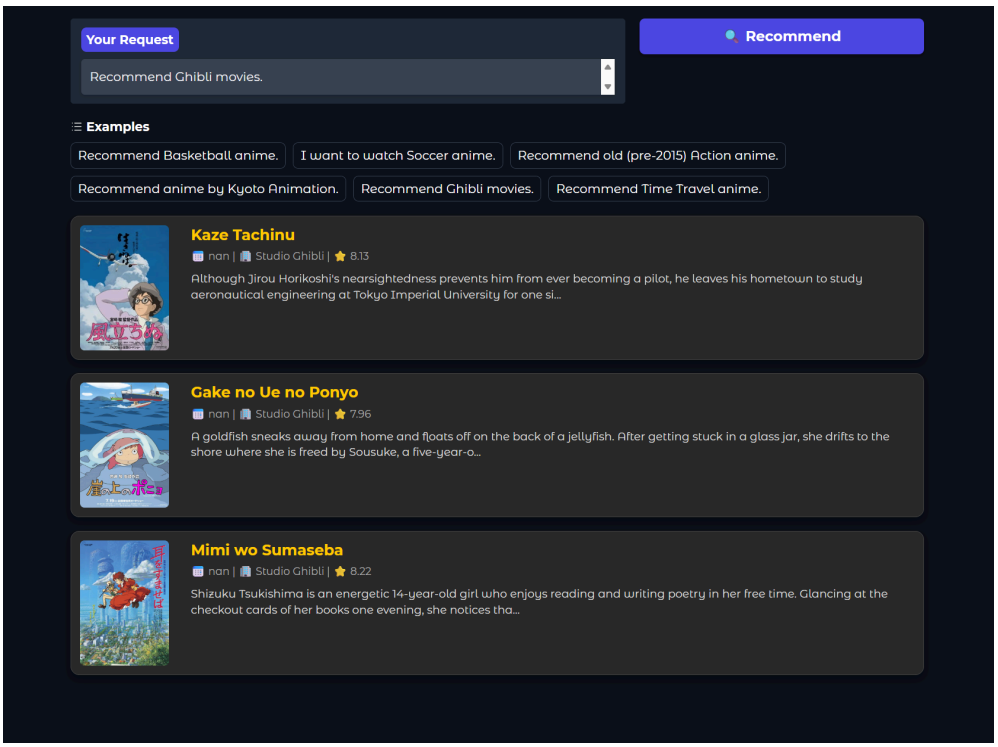


其他成果展示

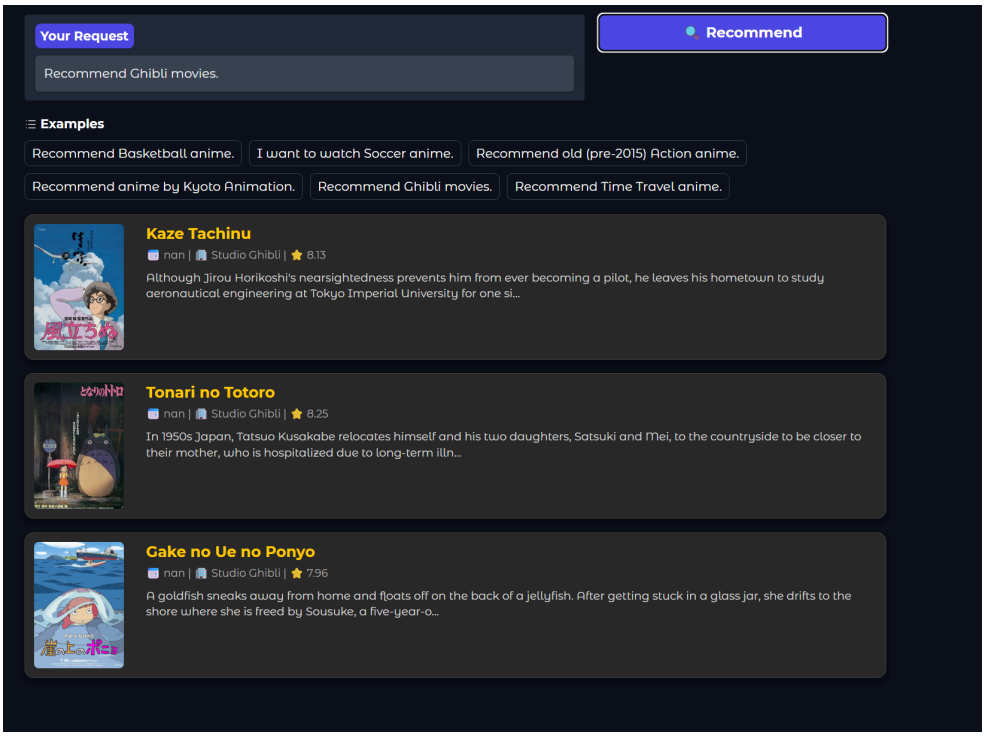
同時查新舊和種類:



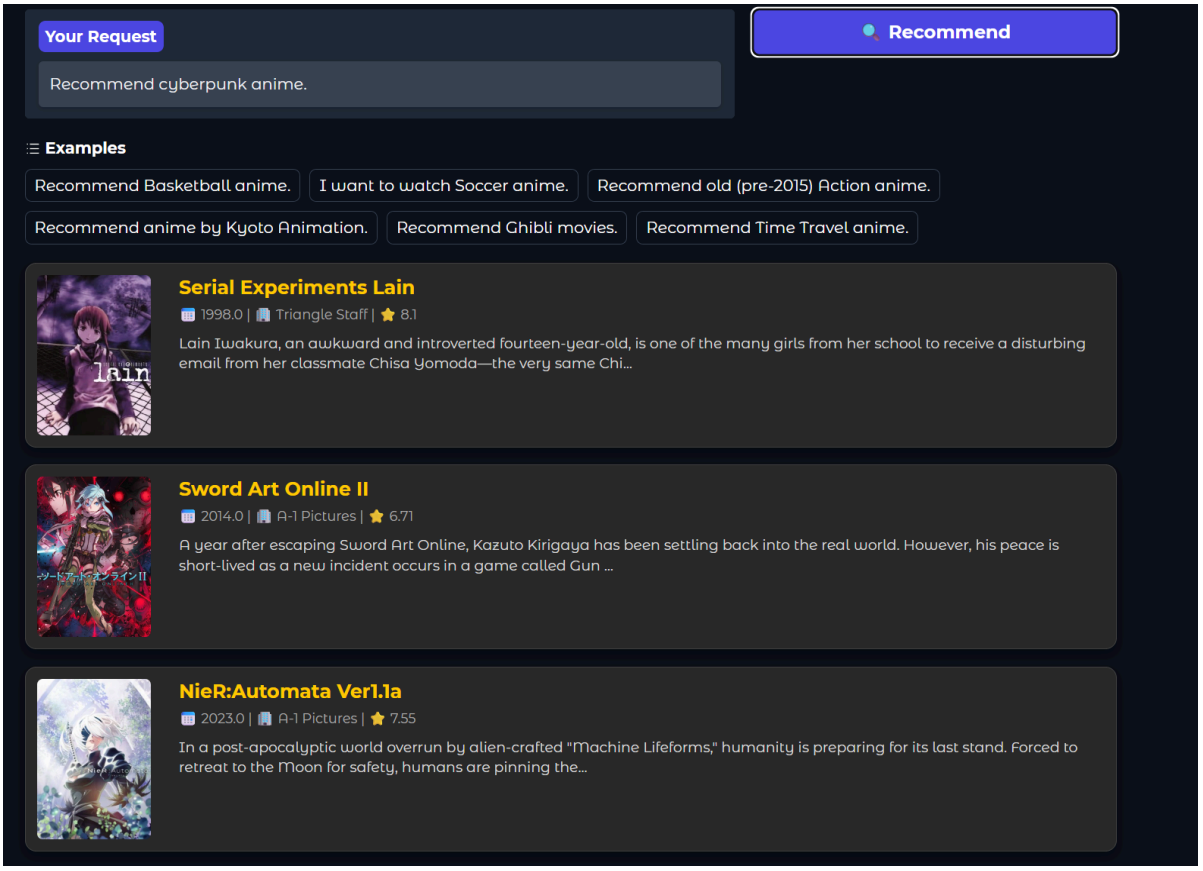
查是哪家工作室的:



補充：每次按下查詢不會是完全相同的結果，具有隨機性



Cyberpunk：



6. Summary and Conclusion

這次的final project成功做出了一套結合 SFT 與 RAG 的動漫推薦系統。透過對 Llama 3.2 3B 模型進行特定領域的微調，我們達成了以下成果：

格式正確以及模型穩定：SFT 模型解決了原始 Base Model 輸出格式混亂且夾雜閒聊的問題。微調後的模型能 100% 穩定輸出 JSON 格式，確保了後端系統檢索的成功率，完全消除了系統崩潰（橘字報錯）的現象。

領域知識的深度挖掘：透過針對資料庫簡介（Description）的 Data Mining，模型成功學習到標籤之外的隱含特徵。實驗證明，微調後的模型能挖掘出如「包含棒球劇情的戀愛番」等跨類型的精準推薦，展現出比單純關鍵字檢索更強的語意理解能力。

我們也利用 LoRA 技術與 4-bit 量化，模型在經過 3 個 Epochs 的訓練後，Loss 曲線穩定收斂至 0.2 以下。驗證損失與訓練損失同步下降且未出現反彈，沒有overfit。

同時，在推論參數上，我們將 Temperature 設定為 0.3 並啟用 do_sample。此低溫設定保證了輸出內容的高度可靠性，同時利用採樣機制為相同查詢提供了具備多樣性的推薦結果。

心得

因為對Gen AI非常有興趣，也很重視這門課，而且老師跟助教人也都很好，所以我們都很認真做這個final project，在定下題目後我們每個人就開始不斷嘗試訓練，不過前面弄幾個都沒得到好結果，但是也累積了很多經驗，透過組員之間不斷地交流，最後不僅大家一起完成了這次的final project，也讓我學到，不能只靠單打獨鬥，還有就算做出失敗的東西也要跟組員交流心得，這樣大家都可以共享彼此的經驗，所以我覺得這比一般的分工還要有效果，在提高效率的同時每個人都能學到更多東西！

7. Future works

這次我們成功做出了SFT 結合檢索的動漫推薦模型。為了進一步提升系統的實用性等，未來可以在兩個方向上做改進：

1. 向量化語意搜尋

目前限制是，目前的檢索層依賴 Python 的關鍵字比對 (Keyword Matching)。雖然精準，但若使用者使用抽象形容詞（如：「我想看一部探討人性黑暗面的作品」），傳統關鍵字搜尋難以命中。

改進方案是可以引入 Sentence-BERT 等開源模型，將資料庫中的Description轉換為高維向量，並使用 Cosine Similarity 進行計算。

預期的結果：這能讓系統跨越關鍵字的限制，真正能理解語意。即使片名或簡介中沒有完全一樣的單字，系統也能根據語意相似性推薦出正確的作品。

2. 使用者回饋Feedback與持續學習

目前的模型是靜態的 (Static)，訓練完成後參數即固定，無法根據使用者的喜好動態調整。改進方法是我們可以在前端介面加入「按讚/按倒讚 (Thumbs up/down)」機制。收集使用者的滿意度數據，建立一個 Preference Dataset 。未來就可利用這些真實回饋數據進行 Direct Preference Optimization或進一步的微調，讓模型隨著使用人數增加，推薦邏輯越來越貼近真實用戶的口味。

3. 加權混合檢索

目前的 RAG 邏輯是「有找到就顯示」，缺乏排序機制。

未來我們可以設計一個 Ranking Algorithm (排序演算法)，綜合考量多個權重：像是SFT 信心度就是模型輸出的順序。

然後還有資料庫、csv裡的屬性：作品的評分 、人氣 (rank 或是 favorites收藏數）等。

這樣就不再只是單純顯示前三筆，而是能動態計算出「最值得推薦」的 Top 3。例如在同樣符合條件的情況下，優先推薦評分較高或較新的作品，提升推薦品質。